League of Legends Diamond Ranked Games

Justin Mundell

12/2/2022

Introduction

League of Legends is one of the most popular video games of all time, with a third-party website estimating the active player count to be around 150 million. The focus of the game is to destroy the opposing team's nexus. If accomplished, the player will win the game. Along with their massive player base, the game contains the most popular Esports, with tournaments taking 4 of the top 5 most watched Esports tournaments ever, according to escharts.com. However, despite the game being developed in the United States, American teams manage to have subpar performances at the tournaments. During the most recent world championship, American teams managed a record of 3 wins and 15 losses. This is the worst record in the competition history of any region that is allowed to bring 3 teams.

The purpose of our data analysis is to learn what factors affect the probability of a team winning a game. It could possibly bring some insight into why the American teams have performed poorly. The main question we want to answer is: What's the chance the blue team has at winning a game during the first 10 minutes, and what explanatory variables are most important in determining this outcome?

Data Description

The data set we used is from Kaggle and is called "high_diamond_ranked_10min". It was sourced from the official League of Legends API and does not have any missing values. According to the description, it contains data during the first 10 minutes of 10,000 ranked games in which the players were from Diamond 1 to Master. For reference, Diamond 1 and Master are some of the highest ranks in competitive gameplay. Only a total of 0.44% of the player base are in those ranks.

Furthermore, we're going to discover which variables help a team win a game. Thus, data taken from complete games will skew heavily in favor of the winning team. An important detail in the game is that gold is a resource that's used to buy items that make characters stronger. In-game experience is used to level up characters, making their abilities more powerful.

Variables Used:

- 'gameID': the unique ID of the game given by Riot
- 'blueWin': A binary variable that is 1 if the blue team won, and 0 if the blue team lost (meaning the red team won).
- 'blueWardsPlaced' and 'redWardsPlaced': The number of wards each team placed to be able to see where enemies are.

- ‘blueFirstBlood’ and ‘redFirstBlood’: A binary variable stating if the blue or red teams scored the first kill onto an opposing team’s player that gives extra gold
- ‘blueDeaths’ and ‘redDeaths’: The number of deaths accumulated for each team
- ‘blueAssists’ and ‘redAssists’: The number of assists a team has
- ‘blueEliteMonsters’ and ‘redEliteMonsters’: The number of elite monsters each team has slain. Elite monsters include dragons, rift heralds, and Baron Nashor (who does not spawn in the first 10 minutes).
- ‘blueDragons’ and ‘redDragons’: The quantity of dragons each team has slain that gives a small buff to either team
- ‘blueHerald’ and ‘redHeald’: The number of Rift Heralds a team has slain
- ‘blueTowersDestroyed’ and ‘redTowersDestroyed’: The number of towers each team has destroyed (towers protect a team, preventing the Nexus from being destroyed)
- ‘blueAvgLevel’ and ‘redAvgLevel’: The average level of the players on each team.
- ‘blueTotalExperience’ and ‘redTotalExperience’: The total experience each team has acquired.
- ‘blueTotalMinionsKilled’ and ‘redTotalMinionsKilled’: The total number of minions killed by each team
- ‘blueJungleMinionsKilled’ and ‘redJungleMinionsKilled’: The number of jungles monsters each team has killed (gives experience and gold to players)
- ‘blueExperienceDiff’ and ‘redExpierenceDiff’: The difference in total experience between each team.
- ‘blueCSPerMin’ and ‘redCSPerMin’: The “creep score” a team earns per minute on average. It’s a metric for how many minions and minion equivalent monsters are slain.
- ‘blueGoldDiff’ and ‘redGoldDiff’: The difference between each team’s total gold number.

Since the data is from the official Riot API, there are no missing values. There are 39 total variables and 9789 total observations. However, this does not mean the data can be used just yet. Variable ‘gameID’ is just an identifier and will not have any bearing on the data, so the column is set to NULL to remove it from the data set. The as.factor function is then used on ‘blueWins’ to turn it into a categorical variable. Other binary variables such as ‘blueFirstBlood’ and ‘redFirstBlood’ were also converted into categorical variables.

During the first ten minutes of a match, it’s only possible to take more than 1 dragon and rift herald. This is because the first dragon spawns in 5 minutes and takes 5 minutes to respawn afterward. The same situation occurs for the rift herald. For this reason, we decided to count them as categorical variables as well.

```
myfile=read.csv("high_diamond_ranked_10min.csv")
high_diamond_ranked_10min = data.frame(myfile)
attach(high_diamond_ranked_10min)

high_diamond_ranked_10min$blueWins=as.factor(blueWins)
high_diamond_ranked_10min$blueFirstBlood=as.factor(high_diamond_ranked_10min$blueFirstBlood)
high_diamond_ranked_10min$redFirstBlood=as.factor(high_diamond_ranked_10min$redFirstBlood)
high_diamond_ranked_10min$blueDragons=as.factor(high_diamond_ranked_10min$blueDragons)
high_diamond_ranked_10min$redDragons=as.factor(high_diamond_ranked_10min$redDragons)
high_diamond_ranked_10min$blueHeralds=as.factor(high_diamond_ranked_10min$blueHeralds)
high_diamond_ranked_10min$redHeralds=as.factor(high_diamond_ranked_10min$redHeralds)
high_diamond_ranked_10min$redGoldDiff=NULL
high_diamond_ranked_10min$redDeaths=NULL
high_diamond_ranked_10min$blueDeaths=NULL
high_diamond_ranked_10min$blueGoldPerMin=NULL
high_diamond_ranked_10min$redGoldPerMin=NULL
high_diamond_ranked_10min$blueCSPerMin=NULL
high_diamond_ranked_10min$redExperienceDiff=NULL
high_diamond_ranked_10min$redCSPerMin=NULL
high_diamond_ranked_10min$gameId=NULL
GameData = high_diamond_ranked_10min
```

Logistic Regression Model

Due to the response variable ‘blueWins’ being categorical, the logistic regression model would be the easiest approach when analyzing the data. The approach has several advantages of being able to test a large amount of data with ease. However, a disadvantage is if our data doesn’t make the model linear since it would make the model prone to overfitting. To combat this disadvantage, removing irrelevant predictors to improve prediction accuracy could be necessary.

After removing some predictors, the process begins by fitting a preliminary logistic regression model on the dataset, comparing the 31 predictors against ‘blueWins’.

$$p(bluewins = 1) = \frac{exp(\beta_0 + \beta_1 * X1 + \beta_2 * X2 \ldots \beta_i * Xi)}{1 + exp(\beta_0 + \beta_1 * X1 + \beta_2 * X2 \ldots \beta_i * Xi)}$$

in which p(blueWins=1) represents a team’s chances of winning, β_0 represents the model intercept, Xi for i = 1, 2, . . ., 31 represents predictors with β_i representing their respective coefficients.

```
glm.fit =glm(blueWins~., data = GameData, family = "binomial")
summary(glm.fit)
```

```
##
## Call:
## glm(formula = blueWins ~ ., family = "binomial", data = GameData)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.7067  -0.8735  -0.1414   0.8671   2.7618
##
## Coefficients: (5 not defined because of singularities)
##                              Estimate Std. Error z value Pr(>|z|)
## (Intercept)                 -3.843e-01  1.242e+00  -0.310 0.756928
## blueWardsPlaced             -2.050e-03  1.340e-03  -1.530 0.126014
## blueWardsDestroyed           8.186e-04  1.145e-02   0.071 0.943031
## blueFirstBlood1              6.959e-02  5.249e-02   1.326 0.184901
## blueKills                   -3.209e-02  3.057e-02  -1.050 0.293819
## blueAssists                 -1.509e-02  1.163e-02  -1.298 0.194401
## blueEliteMonsters            4.056e-02  6.508e-02   0.623 0.533144
## blueDragons1                 3.180e-01  9.354e-02   3.400 0.000674 ***
## blueHeralds1                        NA         NA      NA       NA
## blueTowersDestroyed         -2.539e-01  1.452e-01  -1.749 0.080215 .
## blueTotalGold                2.742e-05  9.902e-05   0.277 0.781818
## blueAvgLevel                 1.021e-02  1.826e-01   0.056 0.955396
## blueTotalExperience         -3.747e-05  8.109e-05  -0.462 0.644047
## blueTotalMinionsKilled      -4.206e-03  1.946e-03  -2.162 0.030639 *
## blueTotalJungleMinionsKilled  2.729e-03  3.333e-03   0.819 0.412777
## blueGoldDiff                 5.012e-04  6.945e-05   7.217 5.33e-13 ***
## blueExperienceDiff           2.655e-04  6.062e-05   4.379 1.19e-05 ***
## redWardsPlaced              -1.541e-03  1.314e-03  -1.173 0.240902
## redWardsDestroyed           -2.635e-03  1.149e-02  -0.229 0.818655
## redFirstBlood1                      NA         NA      NA       NA
## redKills                     2.801e-02  3.064e-02   0.914 0.360642
## redAssists                   1.488e-02  1.136e-02   1.310 0.190163
## redEliteMonsters            -8.361e-02  6.966e-02  -1.200 0.230042
## redDragons1                 -2.045e-01  9.669e-02  -2.115 0.034423 *
## redHeralds1                         NA         NA      NA       NA
## redTowersDestroyed           3.903e-01  1.523e-01   2.562 0.010394 *
## redTotalGold                        NA         NA      NA       NA
## redAvgLevel                 -1.738e-02  1.828e-01  -0.095 0.924228
## redTotalExperience                  NA         NA      NA       NA
## redTotalMinionsKilled        5.409e-03  1.924e-03   2.812 0.004925 **
## redTotalJungleMinionsKilled  6.230e-03  3.326e-03   1.873 0.061049 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
```

```
##     Null deviance: 13695  on 9878  degrees of freedom
## Residual deviance: 10411  on 9853  degrees of freedom
## AIC: 10463
##
## Number of Fisher Scoring iterations: 4
```

```
#calculate R-squared for model
print("R-squared")
```

```
## [1] "R-squared"
```

```
with(summary(glm.fit), 1 - deviance/null.deviance)
```

```
## [1] 0.2398242
```

There are many insignificant predictors found in this summary. The use of step-wise regression to determine the best predictors for the model will fix this issue.

Step Wise regression

```
step(glm.fit)
```

After running the summary for the step-wise regression, it's shown there are 14 important predictors out of 31 total.

```
Start:  AIC=10447.25
blueWins ~ blueWardsPlaced + blueFirstBlood + blueKills + redKills +
    blueDragons + blueTowersDestroyed + blueTotalMinionsKilled +
    blueGoldDiff + blueExperienceDiff + redEliteMonsters + redDragons +
    redTowersDestroyed + redTotalMinionsKilled + redTotalJungleMinionsKilled

                                Df Deviance   AIC
<none>                               10417 10447
- blueTowersDestroyed            1    10419 10447
- redEliteMonsters               1    10420 10448
- blueFirstBlood                 1    10420 10448
- blueWardsPlaced                1    10420 10448
- redDragons                     1    10421 10449
- redTotalJungleMinionsKilled    1    10421 10449
- redKills                       1    10422 10450
- blueKills                      1    10423 10451
- blueTotalMinionsKilled         1    10425 10453
- redTowersDestroyed             1    10425 10453
- redTotalMinionsKilled          1    10428 10456
- blueDragons                    1    10450 10478
- blueExperienceDiff             1    10493 10521
- blueGoldDiff                   1    10544 10572
```

Test error for the logistic regression

```
glm.fit=  glm(formula = blueWins ~ blueWardsPlaced + blueFirstBlood + blueKills +
   redKills + blueDragons + blueTowersDestroyed + blueTotalMinionsKilled +
   blueGoldDiff + blueExperienceDiff + redEliteMonsters + redDragons +
   redTowersDestroyed + redTotalMinionsKilled + redTotalJungleMinionsKilled,
   family = "binomial", data = GameData)
```

```
#calculate McFadden's R-squared for model
print("R-squared")

## [1] "R-squared"

with(summary(glm.fit), 1 - deviance/null.deviance)

## [1] 0.2393485

glm.pred = predict(glm.fit,type = "response")
yhat=glm.pred >0.5
table(GameData$blueWins,yhat)

##    yhat
##     FALSE TRUE
##   0  3634 1315
##   1  1315 3615
```

The test error from the logistic regression is relatively high at 26.62%. For the mathematical computation: ((1315+1315))/(1315+1315+3634+3615)). Our R^2 is also low since only 23.93% of the variation in the probability of the blue team winning can be explained by the model. This indicates that our model is not a very good fit to make an accurate prediction.

Leave-One-Out Cross-Validation

```
set.seed(1)
error <- rep(0, dim(GameData)[1])
for (i in 1:dim(GameData)[1]) {
   fit.glm <- glm(formula = blueWins ~ blueWardsPlaced + blueFirstBlood + blueKills + blueDr
agons + blueTowersDestroyed + blueTotalMinionsKilled +
   blueGoldDiff + blueExperienceDiff + redEliteMonsters + redDragons +
   redTowersDestroyed + redTotalMinionsKilled + redTotalJungleMinionsKilled,
   family = "binomial", data = GameData[-i,])
   pred.up <- predict.glm(fit.glm, GameData[i, ], type = "response") > 0.5
   true.yes <- GameData[i, ]$blueWins == "1"
   if (pred.up != true.yes)
      error[i] <- 1
}
print("LOOCV error")

## [1] "LOOCV error"

mean(error)

## [1] 0.2685494
```

Leave-One-Out Cross-Validation (LOOCV) was run to see if the LOOCV test error would improve the original test error. The LOOCV test error rate was 26.85% which is slightly worse than the original test error rate. This means that LOOCV did not improve the test accuracy.

K-Fold Cross Validation

```
library(boot)
set.seed(3)
cv.error.10 = rep(0,5)
cv.error = rep(0,5)
for (i in 1:5){
  league.glm =  glm(formula = blueWins ~ blueWardsPlaced + blueFirstBlood + blueKills +
    redKills + blueDragons + blueTowersDestroyed + blueTotalMinionsKilled +
    blueGoldDiff + blueExperienceDiff + redEliteMonsters + redDragons +
    redTowersDestroyed + redTotalMinionsKilled + redTotalJungleMinionsKilled,
    family = "binomial", data = GameData)

  cv.error.10[i] = cv.glm(GameData,league.glm,K=10)$delta[1]
}
cv.error.10

## [1] 0.1775665 0.1777092 0.1776097 0.1775702 0.1775591
```

After running the k-fold cross-validation, the test error was reduced to 0.17. Although the test accuracy improved with the same model, it was not high enough to make an accurate prediction.

Bootstrap

```
set.seed(1)
boot.fn <- function(g) {
  library(boot)
  d.glm = glm.fit=  glm(formula = blueWins ~ blueWardsPlaced + blueFirstBlood + blueKills +
    redKills + blueDragons + blueTowersDestroyed + blueTotalMinionsKilled +
    blueGoldDiff + blueExperienceDiff + redEliteMonsters + redDragons +
    redTowersDestroyed + redTotalMinionsKilled + redTotalJungleMinionsKilled,
    family = "binomial", data = g)
  cv.glm (g, d.glm)$delta

}
boot.fn(GameData)

## [1] 0.1775791 0.1775790
```

After running a bootstrap function, the standard error was 0.177. Since the variability isn't high, this model and its set of observations are reliable. The standard error indicates that the sample mean is close to the true mean of the overall sample population.

The success rate of the model was only 73.38%. This isn't entirely sufficient to predict the probability of the blue team winning. The model isn't quite accurate due to the test error rate being relatively high. Additionally, the R^2 based on the test data is low. Although the logistic regression model didn't have high accuracy, we can see which predictors have a significant impact in determining the outcome of a game.

From the logistic regression model, we can conclude that predictors 'blueGoldDiff', 'blueExperienceDiff', 'blueDragons', and 'redTowersDestroyed' have the highest positive effect

on ‘blueWins’. Predictors ‘blueKills’, ‘blueDeaths’ are also significant. Further, other predictors such as ‘redTotalJungleMinionsKilled’, ‘redDragons’, and ‘blueWardsPlaced’ aren’t that significant from an alpha = 5%, level but were still included in the model.

Oddly, the predictor ‘redTotalMinionsKilled’ has a positive effect on ‘blueWins’ while ‘blueTotalMinionsKilled’ has a negative effect. They may be extremely significant, but these signs are not the coefficient signs we expected since ‘blueTotalMinionsKilled’ should be positive. Logically, the more minions the blue team kills, the more gold they will get which should increase the chances of them winning. This is the opposite of what was expected.

After running the cross-validations on the logistic regression model, the K-fold cross-validation performed the best with a test error rate of 17%. This is much better than the original test error of 26.62%. Additionally, it’s also better than the LOOCV test error of 26.85%. The validation set approach was not used because it would perform worse than the cross-validation methods. It makes sense that K-fold cross-validation performed better than LOOCV since it doesn’t lose in estimation quality and estimate efficiency to LOOCV. Also, the LOOCV method is more computationally expensive in comparison to K-fold. On another note, it’s surprising that the original test error rate performed slightly better than LOOCV.

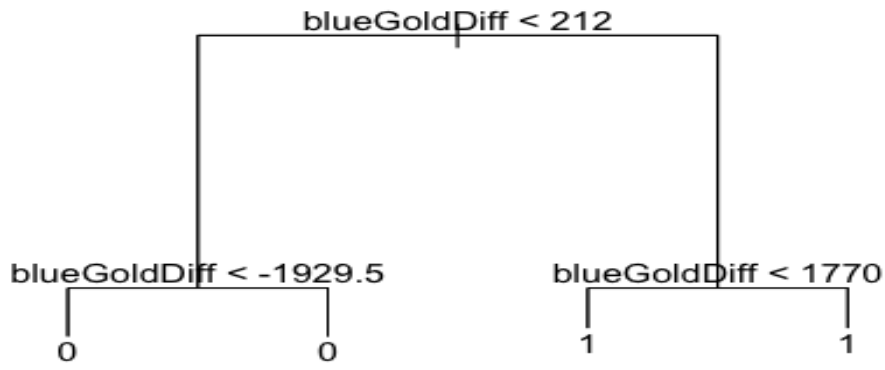
Decision Tree

This section of the analysis will focus on tree-based methods such as decision trees and random forests. Tree methods have a distinct advantage over logistic regression, as they can be applied to both classification and regression problems, while logistic regression can only handle classification. Another advantage of the trees is that it’s easier to interpret through tree diagrams since they give a nice visual overview of the results. However, they have a lower prediction accuracy than other methods. Luckily, techniques such as decision trees, bagging, random forests, and boosting can be used to increase accuracy and reduce variance.

```
library(tree)
tree.ranked=tree(blueWins~.,GameData)
summary(tree.ranked)

##
## Classification tree:
## tree(formula = blueWins ~ ., data = GameData)
## Variables actually used in tree construction:
## [1] "blueGoldDiff"
## Number of terminal nodes:  4
## Residual mean deviance:  1.101 = 10870 / 9875
## Misclassification error rate: 0.2747 = 2714 / 9879

plot(tree.ranked)
text(tree.ranked, pretty = 0)
```



As seen, 4 terminal nodes are produced from this decision tree. The training error rate is 27.47% From the tree plot, it shows that the predictors 'blueGoldDiff' matters the most. This was expected, as most of the predictors in the data set are related to gold. The first step in evaluating a tree-based method is to create a training set and a testing set. This is to make sure that the data is not overfitted to the entire data set. In this case, the training set will be 80% of the data set, while the testing set will be the remaining 20%. The decision tree will be trained on the training set, and the accuracy will be tested on the test set to determine the performance of the model.

```
set.seed(10)
train=sample(1:nrow(GameData),nrow(GameData)*0.8)
ranked.test=GameData[-train,]
Wins.test=blueWins[-train]
tree.ranked=tree(blueWins~.,GameData,subset=train)
tree.pred=predict(tree.ranked,ranked.test,type="class")
table(tree.pred,Wins.test)
```

Test error rate for set.seed(1-10) in order: 27.58%, 28.64%, 29.50%, 28.89%, 26.82%, 28.39%, 27.73%, 27.53%, 27.07%, and 28.08%

After averaging the 10 different test error rates iterations, the average test error rate for the decision trees is 28.02%. In the tree diagram, we can see two splits that lead to the same terminal node. These splits exist to increase node purity but does not decrease the MSE.

Pruning the Tree

```
set.seed(3)
cv.ranked=cv.tree(tree.ranked,FUN=prune.misclass)
cv.ranked

## $size
## [1] 4 2 1
##
```

```
## $dev
## [1] 2237 2237 4067
##
## $k
## [1] -Inf    0 1727
##
## $method
## [1] "misclass"
##
## attr(,"class")
## [1] "prune"         "tree.sequence"
```

The tree is pruned to see if we can avoid overfitting the data to the training set. Despite the cross-validation error rate being the same for both trees, the 2 node trees visually look better.

The $dev corresponds to the cross-validation (CV) error rate. The lowest CV error is 2219 for both sizes 4 and 2.

```
prune.ranked=prune.misclass(tree.ranked,best=2)
plot(prune.ranked)
text(prune.ranked,pretty=0)
```

```
tree.pred=predict(prune.ranked,ranked.test,type="class")
table(tree.pred,Wins.test)

##          Wins.test
## tree.pred   0   1
##         0 811 372
##         1 183 610
```

This obtains almost the same test error rate as the original un-pruned tree at 28.08%. It can be argued that there's no point in pruning the tree due to obtaining the same test errors, but the pruned tree does look better visually in terms of interpreting the nodes. For interpreting the

nodes, if the blue-gold difference is less than 632, the blue team will lose. However, if the blue-gold difference is greater than 632, it's predicted that the blue team would win.

Bagging

Bagging stands for bootstrap aggregating. To reduce the variance of our decision trees, we're able to make a lot of decision trees out of one training set and average all the predictions to create one tree. In the case of a classification tree, a majority vote is taken to find the most commonly occurring prediction. The main downside of the tree aggregation methods is computation time.

```
library(randomForest)

## randomForest 4.7-1.1

## Type rfNews() to see new features/changes/bug fixes.

set.seed(i)
n=nrow(GameData)
p=ncol(GameData)-1
WINS.test=GameData[-train,"blueWins"]
X.test=GameData[-train,-1] #take away blueWins column
bag.model=randomForest(blueWins~.,data=GameData,subset=train,xtest=X.test,ytest=WINS.tes
t,ntree=500,mtry=p,importance=TRUE)
bag.model
```

The following seeds are in order as (28, 19, 38, 43, 22, 85, 51, 73, 90, and 4) with their respective test error rates in order as (27.73%, 27.58%, 27.78%, 27.33%, 27.58%, 27.38%, 27.83%, 27.83%, 27.78%, and 27.48%).

The mean of the 10 test error rate iterations is 27.63%. This number is high, likely due to how complex the game is. It's also an example of how a game cannot be determined by a single variable.

To better understand the data, an "importance plot" can be generated to determine what variables reduce the GINI index the most. The GINI index is a measure of variance in classification trees. A reduction of GINI index means the variable means a lot to the model, and the function varImpPlot results in plots that show the relative reduction in variance and GINI index.

```
varImpPlot(bag.model)
```

As shown from the plot, predictor ‘blueGoldDiff’ is the most important variable by a large margin, which is why it is the only variable that shows up on the tree. The next most important variables are ‘blueTotalDiff’ and ‘blueTotalGold’. These predictors are much less important than ‘blueGoldDiff’.

Random Forests

Random forests are an alternative way to do bagging that changes how each split is done. In random forests, when a split is considered, instead of all the possible variables being considered, only a random subset of predictors is considered instead. For a classification tree, the subset used will have root(p) predictors, where p is the total number of predictors. This is important to reduce variance as it prevents specific variables from dominating all the trees. The random forests should improve bagging since it decorates the bagged trees. The model formula used is the same as the one used in decision trees.

```
set.seed(i)
train = sample(1:nrow(GameData), 0.8*nrow(GameData))
WINS.test=GameData[-train,"blueWins"]
X.test=GameData[-train,-1] #take away blueWins column
rf.model=randomForest(blueWins~.,data=GameData,subset=train,xtest=X.test,ytest=WINS.test,
ntree=500,mtry=sqrt(p),importance=TRUE)
rf.model
```

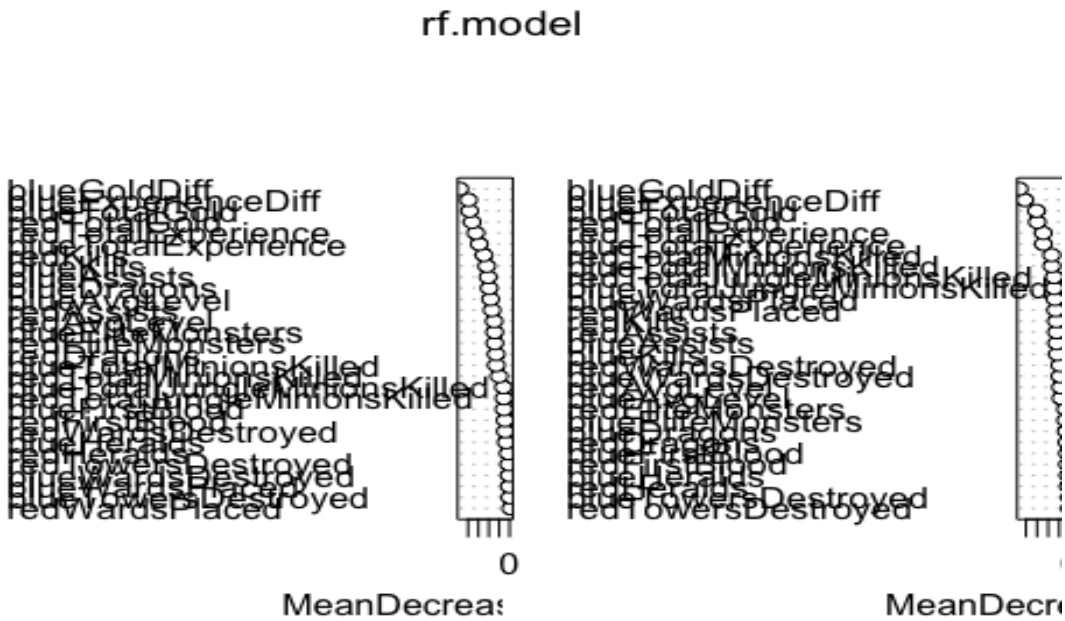
The following seeds are in order as (28, 19, 38, 43, 22, 85, 51, 73, 90, and 4) with their respective test error rates in order as (25.86%, 27.58%, 26.72%, 26.62%, 28.14%, 27.88%, 28.54%, 28.14%, 25.61%, and 27.63%).

The mean of the test error rate over 10 iterations is 27.27% for the random forests. This error rate is remarkably like the error rate of the single decision tree and bagging models. It’s likely that in

the end, the random forests choose gold difference as the most important variable in the majority vote. The varImpPlot affirms this belief.

Random forests usually have better error rates than bagging and single decision trees. This appears to be true in our case since the random forest test error rate is 27.27%, the decision tree test error rate is 28.02%, and the bagging test error rate is 27.63%. It's important to note that the random forest only performed slightly better than the others.

```
varImpPlot(rf.model)
```



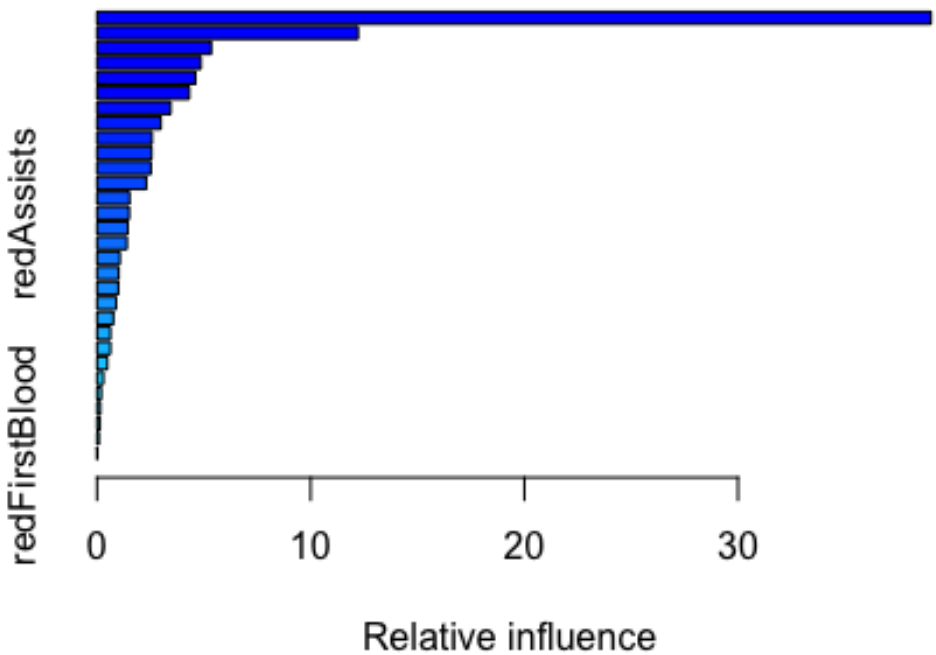
Boosting

The boosting method involves the creation of multiple trees, like random forests and bagging, but instead uses information from the previous tree to create the next. The three parameters boosting rely on are B, which is the number of trees made, λ, known as the shrinkage parameter, and d, the number of splits in each tree. The λ is typically around 0.01 to 0.001, as learning too much from the previous trees may result in very biased data. The boosting function in Rstudio is contained in the gbm library.

```
library(gbm)

## Loaded gbm 2.1.8.1

set.seed(1)
boost.ranked = gbm(as.integer(blueWins) - 1~., data = GameData,shrinkage=0.01,
                   distribution = "bernoulli",
                   n.trees = 5000,
                   interaction.depth = 4,verbose = F)
summary(boost.ranked)
```




```
##                                var     rel.inf
## blueGoldDiff                   blueGoldDiff 39.02827608
## blueExperienceDiff             blueExperienceDiff 12.21568070
## blueTotalGold                  blueTotalGold  5.34459716
## redTotalExperience             redTotalExperience  4.86117704
## redTotalGold                   redTotalGold  4.60221042
## blueTotalExperience            blueTotalExperience  4.31882648
## redTotalMinionsKilled          redTotalMinionsKilled  3.44668182
## blueTotalMinionsKilled         blueTotalMinionsKilled  2.98796104
```

After performing boosting, we're able to see that the variable 'blueGoldDiff' is still the most important variable, with 'blueExperienceDiff' coming in at second. Note that we cut out the remaining variables due to space, including the most important ones in the boosting visualization.

Prediction comparison

For the test error rates for all our models, the logistic regression model performed the best through the K-fold cross-validation method. This method obtained a test error rate of 17%, meaning that the model accurately predicts 83% of the time. The decision trees had a test error rate of 28.02%, the bagging was 27.63%, and the random forest was 27.27%. The tree-based methods did not perform as well as the logistic regression did based on our data set.

Conclusion

Despite the easy visualization and interpretation of the decision trees, they generally don't yield the same level of predictive accuracy as some other classification approaches do. Usually, random forests and bagging assist in this problem, but these two methods only improved the decision trees' test error rate slightly. This is relatively surprising since random forests and bagging should improve the errors by quite more. The other classification method, logistic regression, outperformed every method. For the logistic regression model to achieve the best predictive model, K-fold cross-validation had to be used to get the lowest test error rate of 17%. So, it's rather difficult to conclude that this is the most accurate model when predicting the probability of a team winning due to some possible issues in the data set.

There are limitations to determining the chances of a team winning during the first ten minutes of the game. This means that potential details are missing that could possibly help explain why our predictive accuracy is not that high. Thus, having information on the types of characters these players in the data set used would help in determining how some characters are specifically designed to be weak in the earlier stages of the game (and thus fall behind in gold) but become the strongest after around 30 minutes. Not considering the existence of these details will inevitably result in large errors.

Bibliography

League of Legends. The Game Statistics Authority: ActivePlayer.io. (2022, September 7). Retrieved December 2, 2022, from https://activeplayer.io/league-of-legends/

Esports teams and events stats: Dota 2, LOL, CS:GO | Esports Charts. (n.d.). Retrieved December 2, 2022, from https://escharts.com/tournaments

Ma, Y. L. (2020, April 13). League of legends diamond ranked games (10 min). Kaggle. Retrieved December 2, 2022, from https://www.kaggle.com/datasets/bobbyscience/league-of-legends-diamond-ranked-games-10-min